49274 — Space Robotics

Team Project: Exploring a Martian Cave

Autonomous perception, planning and science for a Mars rover

Team	Andrew Than, Christina Li, Karan Sharma
Subject	49274 Space Robotics
Institution	University of Technology Sydney
Date	October 2026

This report describes a ROS 2 / Gazebo system in which a simulated Mars rover explores an unknown cave, detects and localises artefacts, inspects them, analyses the cave geometry, and samples a real NASA water-concentration map adaptively.

Contents

1 Group Member Details **2**

2 Generative AI Disclosure **2**

3 Statement of Individual Contributions **3**

4 Project Overview **3**

5 Perception **4**
5.1 Perception 1–3 (Andrew) 4
5.2 Additional artefact detectors and their scoring (Karan) 4

6 Planning **6**
6.1 Planning 1 — Autonomous exploration 6
6.2 Planning 2 — Close-range inspection 6
6.3 Planning 3 — Behaviour switching 6

7 Advanced 2 — Cave Geometry Analysis **6**
7.1 What the task asks for 6
7.2 The idea in one picture 7
7.3 How a narrow spot is tested 8
7.4 The data going in and out 8
7.5 Results 8
7.6 Limits and possible improvements 9

8 Advanced 6 — Adaptive Scientific Sampling **10**
8.1 Objective 10
8.2 The science data: real NASA measurements of water in Martian soil 10
8.3 How the data was put onto the cave map 11
8.4 What the data looks like in a simulator run 13
8.5 The real instrument and our virtual one 13
8.6 What is real and what is not 15
8.7 Method 15
8.8 Results 17
8.9 Discussion and limits 19

9 Results **19**

10 Reflection on Teamwork **20**

References **21**

1 Group Member Details

Name	Student number	Main responsibility
Andrew Than	**[TODO: Andrew: student number]**	Perception (tasks P1–P3)
Christina Li	**[TODO: Christina: student number]**	Planning (tasks L1–L3)
Karan Sharma	26276485	Advanced tasks A2 and A6; report lead; repository

Task labels: P1–P3 are Perception 1–3, L1–L3 are Planning 1–3, and A2 and A6 are Advanced 2 (cave geometry analysis) and Advanced 6 (adaptive scientific sampling).

2 Generative AI Disclosure

STUDENT ACTION REQUIRED — every member

The paragraph for Karan below states what the AI tool did, from the record of the work. Karan must add the closing verification statement. Andrew and Christina must write their own paragraphs, truthfully, in the same form: which tool, what for, what not for, and what you can explain unaided. In Assessment task 2 the marker flagged a vague declaration and warned of a viva, so every member should expect to explain any line they submit.

Karan Sharma. Tool: Claude Code (Anthropic), a command-line coding assistant, used throughout.

Used for:

- Writing first versions of the cave geometry code (`cave_geometry.py` and its ROS node), the adaptive sampling code (measurement field, Gaussian Process sampler and ROS node), the offline test scripts, and the colour and ice detectors with their scoring scripts.
- Locating and downloading the NASA Mars Odyssey data set, and writing the code that reads it.
- Running the simulator and the offline experiments, finding faults in them (false goal failures, a rover that became wedged, an unstable report of the narrowest passage), and proposing and applying fixes.
- Drafting the project plan, responsibilities, dependency list, repository structure and this report.

Not used for: the decision to use real NASA data (Karan's request), the choice to take the hardest task, running the final demonstration, or recording the video. The AI tool proposed the particular pair of Advanced tasks (2 and 6) and the task split in the project plan; Karan accepted them.

Verification. Each component was run in the simulator and scored against known truth (the artefact positions and cave walls in the world file) before being accepted. All numbers in this report were produced by the scripts in the repository. **[TODO: Karan: add a sentence stating honestly which parts of the geometry and sampling code you can reproduce and explain unaided, and which you are still consolidating.]**

Andrew Than. **[TODO: Andrew: tool, what it was used for, what not, and what you can explain.]**

Christina Li. **[TODO: Christina: tool, what it was used for, what not, and what you can explain.]**

3 Statement of Individual Contributions

STUDENT ACTION REQUIRED — every member

The "Completed?" entries below are Karan's reading of the repository and must be corrected by the member who owns each row. Anyone who disagrees with the statement may write to the Subject Coordinator privately.

Task	Completed?	Member	Contribution
Perception 1 (dataset)	**[TODO: confirm]**	Andrew Than	Image-saving code in the ROS node.
Perception 2 (detection)	**[TODO: confirm]**	Andrew Than	First colour detector (green objects).
	Yes, additional detectors	Karan Sharma	Alien, crystal, mushroom, sphere and ice detectors, simulator capture and scoring scripts (Section 5).
Perception 3 (localisation)	**[TODO: confirm]**	Andrew Than	Not yet on the shared repository.
Planning 1 (exploration)	**[TODO: confirm]**	Christina Li	Frontier exploration.
Planning 2 (inspection)	**[TODO: confirm]**	Christina Li	Stand-off approach goal and inspection.
Planning 3 (switching)	**[TODO: confirm]**	Christina Li	Explore–inspect state machine with timeout, retry and visited record.
Advanced 2 (geometry)	Yes	Karan Sharma	Design, code, simulator tests, evidence.
Advanced 6 (sampling)	Yes	Karan Sharma	Design, code, simulator tests, comparison.
Report writing	In progress	Karan Sharma (lead), all	Karan edits and unifies; each member writes their own task section.
Recorded videos	**[TODO: confirm]**	**[TODO: who]**	**[TODO: what]**

4 Project Overview

The task is to program a simulated Mars rover to explore an unknown lava-tube-like cave without human input: build a map, find artefacts of scientific interest, and understand the environment. The cave is a Gazebo world of walls and 14 artefacts of five types (green alien, green crystals, white sphere, blue mushrooms, ice formations) plus stop signs. The rover carries a laser scanner, an RGB camera and a depth camera; `slam_toolbox` builds the map and Nav2 plans and drives to goals.

The team split the work into three parts. **Perception** (Andrew) detects and localises artefacts from the camera. **Planning** (Christina) chooses where to go, switches between exploring and inspecting, and keeps a record of visited artefacts. **Advanced** (Karan) analyses the cave geometry as it is explored and samples a spatial science field adaptively.

What was achieved. Status on 12 October, taken from the shared repository and from test runs in the simulator.

- **Advanced 2 and Advanced 6** (Karan) are complete and merged. Both ran in the simulator with real navigation and a real SLAM map: the cave geometry analysis found the widest open area and the narrow passages, and the adaptive sampler took 30 readings of a real NASA water map in each of 12 runs.

- **Perception** (Andrew): an image saver and a first colour detector are on the main branch. Karan also built and scored detectors for the alien, crystals, mushroom, sphere and ice on a separate branch (Section 5). Locating artefacts on the map (Perception 3) was not on the repository at the time of writing.
- **Planning** (Christina): exploration, inspection and behaviour switching are written on the planning branch and are not yet merged into the main branch. Karan has not run them.
- **The parts have not been run together** in one launch.

[TODO: All: update this list to the final state before submission. The brief penalises statements that overstate what was achieved.]

Software layout. All code is in one ROS 2 package, `cave_explorer`, in a shared Git repository, with one folder per part (`perception/`, `planning/`, `advanced/`). Tests that need no simulator and the scripts used for every figure are in `scripts/`. The Advanced tasks run as their own nodes, so they do not edit the shared node that the other parts use.

5 Perception

STUDENT ACTION REQUIRED — Andrew

Andrew writes Perception 1, 2 and 3: approach, how well it worked, how it could be improved, key code, and evidence. Perception 1 needs the number of images collected for each artefact type. Perception 3 needs the method for estimating positions, how repeated detections of one artefact were merged, and RViz screenshots of markers at the artefacts. Karan has only written the part below.

5.1 Perception 1–3 (Andrew)

[TODO: Andrew: write these three subsections.]

5.2 Additional artefact detectors and their scoring (Karan)

[TODO: Karan and Andrew: agree how this subsection is presented, so the report is clear who built what. The text below is a factual record.]

Why a test set was needed. The first detector on the shared repository thresholds green hue. To measure it, and to build better ones, Karan wrote a script that teleports the rover to many viewpoints around each artefact in the simulator and records the camera and depth images (405 images). The world file gives the true position of each artefact, so each image is labelled with which artefact types are really in view (1–9 m away, with a clear line of sight). The labels are used only to find colours and to score detectors; the detectors see only the image.

Colour detectors. Each type has a distinct colour. The alien is pale lime (saturation 160–180), the crystals deep emerald (saturation 204–255), the mushroom caps saturated blue (hue 110–112) and the sphere pale blue-white (hue 105, saturation about 49). A mask per type, cleaned with morphological opening and closing, gives boxes. The rover's own body, which fills the bottom-left of every frame, is blanked out. The alien and crystals are told apart by saturation alone.

Table 1: Artefacts found among those labelled as visible (405 captured images). A box counts when it covers the artefact's true image column. "False alarm" means a box of a type with no artefact of that type ahead within 12 m.

Type	1–3 m	3–5 m	5–9 m	All	False-alarm boxes
Alien (new)	17/17	8/8	8/8	**33/33 (100%)**	13
Alien (main branch)	9/17	1/8	0/8	10/33 (30%)	5
Crystals (new)	19/21	11/11	9/9	39/41 (95%)	8
Crystals (main branch)	15/21	9/11	3/9	27/41 (66%)	9
Mushroom (new)	24/25	11/11	9/10	44/46 (96%)	3
Sphere (new)	17/21	7/9	6/8	30/38 (79%)	3

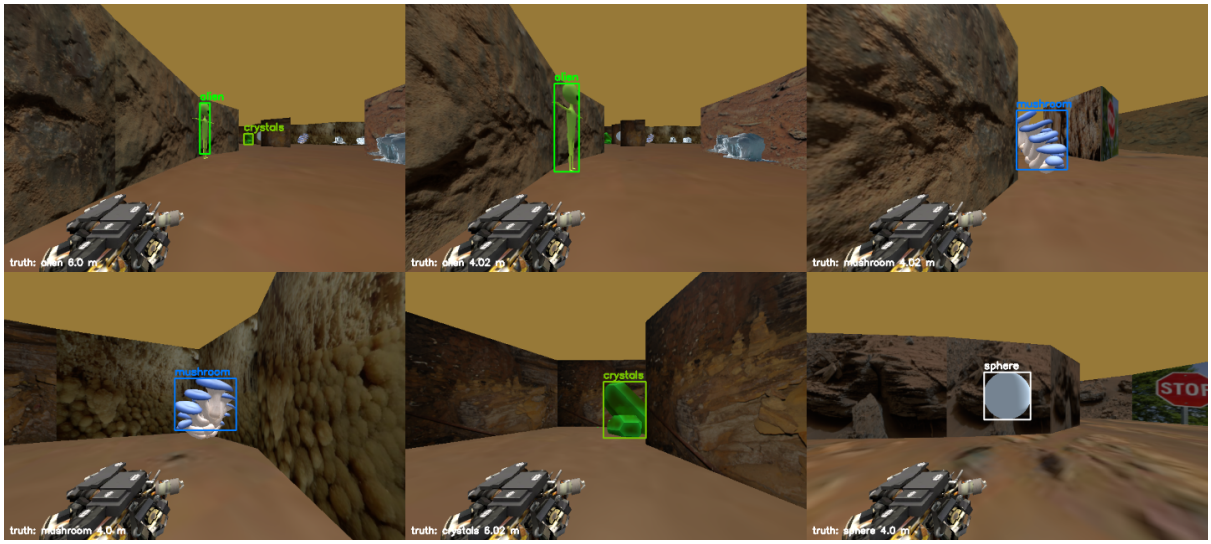


Figure 1: Detections on captured images, with the true artefact type and range written at the bottom of each panel.

Ice needed a different method. Ice is hard for three reasons found by measurement. Each formation is about $5 \times 5 \times 1.5$ m, and the point used by the world file lies up to 3 m from its centre, so a point label was wrong. Its pixels are about half blue-grey and half brown-beige, the same colour as the cave walls and floor, so no colour range works: a learned colour rule finds 15–21% of ice and puts a false box in 62–82% of images. The detector in `ice_detector.py` therefore classifies pixels with a random forest using colour, local texture and two depth features (height above the floor and surface bumpiness). True ice pixels for training and scoring come from projecting the ice mesh into each image and keeping the points whose range agrees with the depth image. Trained on half the images and scored on the other half, it finds ice in 18 of 33 images that show it (55%), with a false ice box in 5% of images, at 62 ms per frame. This is usable but not reliable.

Limits. Colour ranges were measured in this one world and tuned on the same 405 images, so the figures above are optimistic for a new scene or a fresh set of viewpoints. The detectors return a box and image column; turning that into a map position (Perception 3) needs the depth image and the robot pose. The detectors are not yet called from the rover's image callback. The image passed to them must be in BGR order; ROS images arrive as RGB.

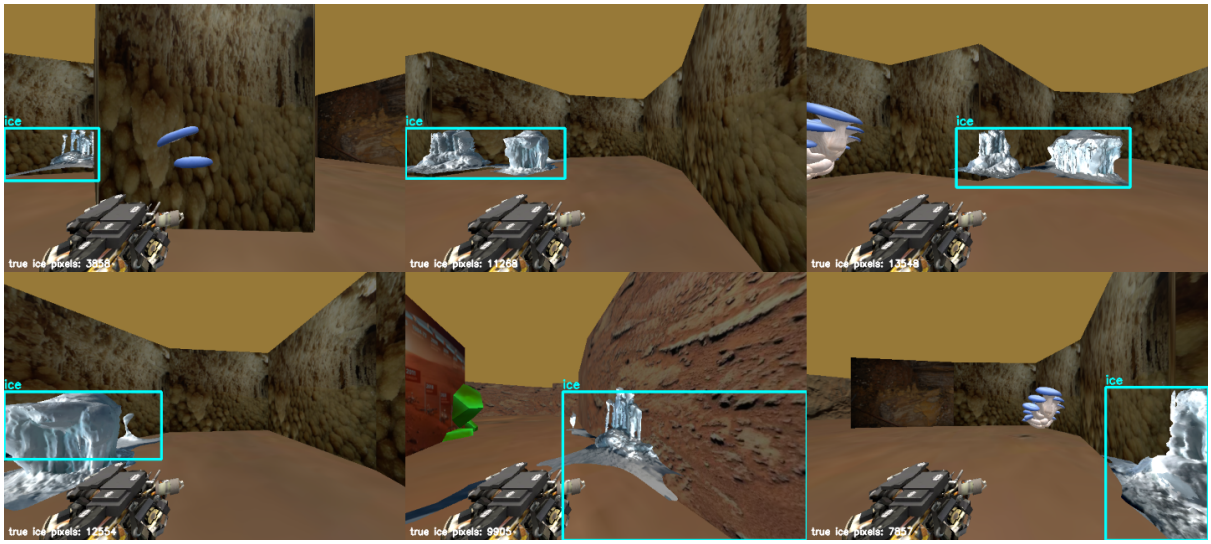


Figure 2: Ice detections on images the forest was not trained on. These are clear views; the overall detection rate is 55%.

6 Planning

STUDENT ACTION REQUIRED — Christina

Christina writes Planning 1, 2 and 3: the exploration method and design choices, how well it works, what she would add with more time; the inspection approach and difficulties; and the switching behaviour, including timeout, retry, abandon and the visited record. Include RViz screenshots. The brief asks for markers showing chosen frontiers and visited artefacts.

6.1 Planning 1 — Autonomous exploration

[TODO: Christina]

6.2 Planning 2 — Close-range inspection

[TODO: Christina]

6.3 Planning 3 — Behaviour switching

[TODO: Christina]

7 Advanced 2 — Cave Geometry Analysis

7.1 What the task asks for

As the rover explores, the program should work out the *shape* of the cave from the map the rover is building. In particular it should find the **widest open area** found so far and a **narrow passage** (a bottleneck), and show both in RViz on top of the map. Wide areas could suit future human activity, and narrow passages matter for navigation and safety. The result is updated every time the map changes.

7.2 The idea in one picture

The method rests on one simple question, asked for every free spot on the map: *how far away is the nearest wall?* Everything else (the centre-line, the widest area, the narrow spots) is read off the answer. Figure 3 shows the six steps on the real map from a Gazebo run, and the text below explains each one.

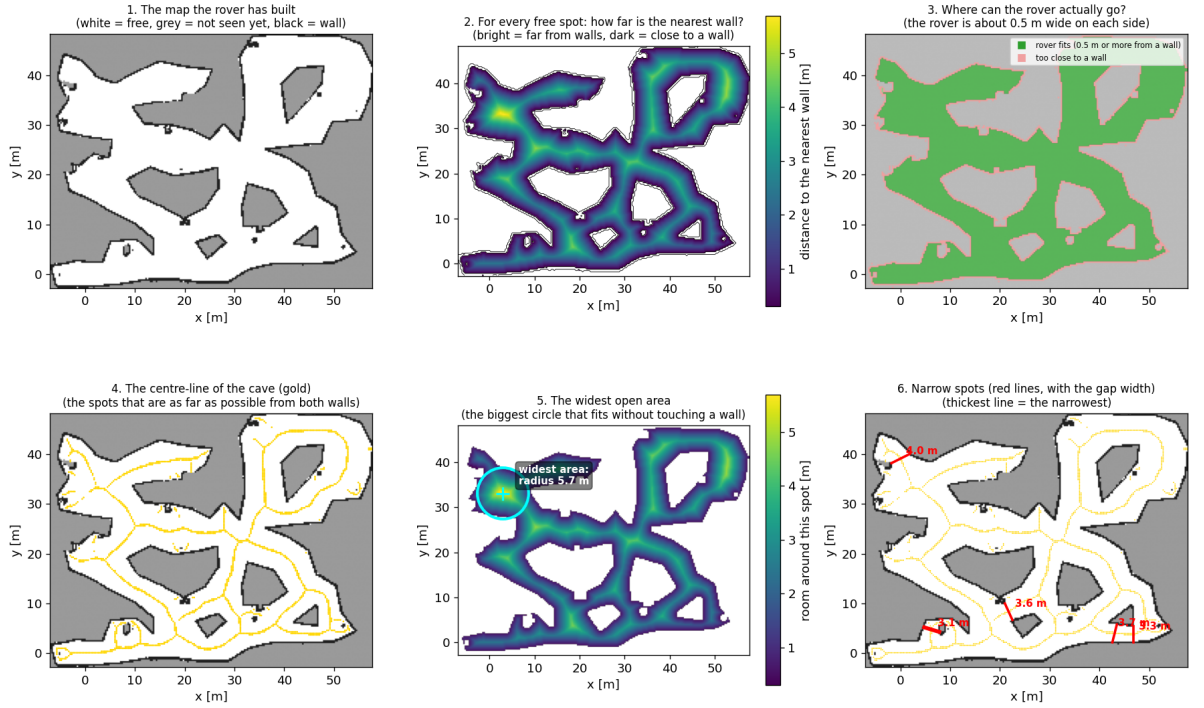


Figure 3: The six steps of the cave geometry analysis, on the map the rover built in Gazebo.

1. **The map.** The rover's map is a grid of small squares, each 0.3 m wide. Every square is marked as free, as a wall, or as not seen yet. We only use what the rover has really seen.
2. **Distance to the nearest wall.** For every free square the program works out how far it is from the closest wall. Squares in the middle of a wide room get large numbers, and squares hugging a wall get small ones. (The technical name for this is a *distance transform*.) It is recalculated whenever the map updates, so it grows with the map.
3. **Where the rover can go.** The rover is about 0.5 m wide on each side, so squares closer than 0.5 m to a wall are ruled out, and so are free areas the rover could not reach from where it is.
4. **The centre-line.** Imagine walking away from a wall. The distance to it grows by exactly one metre for each metre you walk. That stops being true on the centre-line of a corridor: step towards one wall and you move away from the other, so the distance stops growing steadily. The program looks for exactly the spots where the distance stops growing steadily and marks them in gold. They form a line along the middle of every passage. (The technical name is the *medial axis*. The rule uses how fast the distance changes from square to square, its *gradient*.)
5. **The widest open area.** Among all the squares the rover can reach, the program finds the one with the most room around it. The circle that just fits there, touching no wall and no unseen space, is the widest open area, and its radius says how roomy it is. Unseen space is not counted as open.
6. **Narrow spots.** A narrow spot is a place along the centre-line where the gap between

the walls is smaller than nearby, like a doorway between two rooms. Each one is shown as a red line across the gap, labelled with the width. The thickest line is the narrowest. Finding them takes a careful test, described next.

```
def ridge_of(clearance, accessible, resolution, gradient_threshold=0.8):
    d_row, d_col = np.gradient(clearance, resolution)
    return accessible & (np.hypot(d_row, d_col) < gradient_threshold)
```

Listing 1: Step 4 in code: the centre-line is where the distance stops growing by one metre per metre.

7.3 How a narrow spot is tested

A spot on the centre-line where the gap is small is only a *narrow passage* if it really separates something. A small bump on a wall also makes the gap smaller, but it separates nothing. So each candidate gets a *cut test*: draw a line across the gap, from the nearest wall on one side to the nearest wall on the other, and pretend the line is a wall. Then look at the free space around it (within 5 m). If it splits into two separate areas, each at least 3 m^2, and at least one of them opens out to 1.5 times the gap, it counts as a narrow passage. Figure 4 shows one candidate that passes the test and one that fails.

Two more details. Each passage is called a *doorway* if both sides open out and a *corridor* if only one does. The program also checks whether the spot is a *saddle*, like a mountain pass between two wider areas, using the curvature of the distance map. This is reported but not used to decide.

Making the report steady. The map moves by one square between updates. Early on, this made the "narrowest passage" jump between near-equal candidates 31 times in one run. A passage is now shown only if it was also found, within 1.5 m, in the previous update, and a change is announced only if the width moves by more than 0.5 m or the place by more than 3 m. A brief false 1.2 m "doorway", caused by a half-drawn wall, disappeared with this check.

7.4 The data going in and out

7.5 Results

On the map the rover built. The same analysis was run on the map made by `slam_toolbox` during a full Gazebo run (Figure 5). It found a reachable area of about 1500 m^2, the widest open area with a radius of 5.7 m at $(3.0, 33.1)$ (about 100 m^2), and five narrow spots, 3.1, 3.3, 3.6, 3.7 and 4.0 m wide. The widest area is the large chamber in the top left of the map.

It grows as the rover explores. Figure 6 replays the run: the map is shown only around the places the rover had visited so far (within 12 m of each stop), then analysed. After the first stop, the widest area has a radius of 2.8 m. After three stops it is 4.5 m, and after eight it reaches 5.7 m, when the rover reaches the big chamber in the top left. The known free space grows from 82 m^2 to more than 1000 m^2 over the same stops.

Checking against a separate source. To check the answer is sensible, the same analysis was run on a map drawn directly from the wall positions in the simulator's world file, with no simulator running. This is independent of the laser and of SLAM.

The widest area agrees within 0.2 m in radius and 0.7 m in position. Both maps find the same two gaps of about 3.1–3.3 m, near $(6, 4)$ and $(47, 4)$, but they disagree about which is narrowest. The difference is 0.2 m or less, which is smaller than one map square (0.3 m), so it is

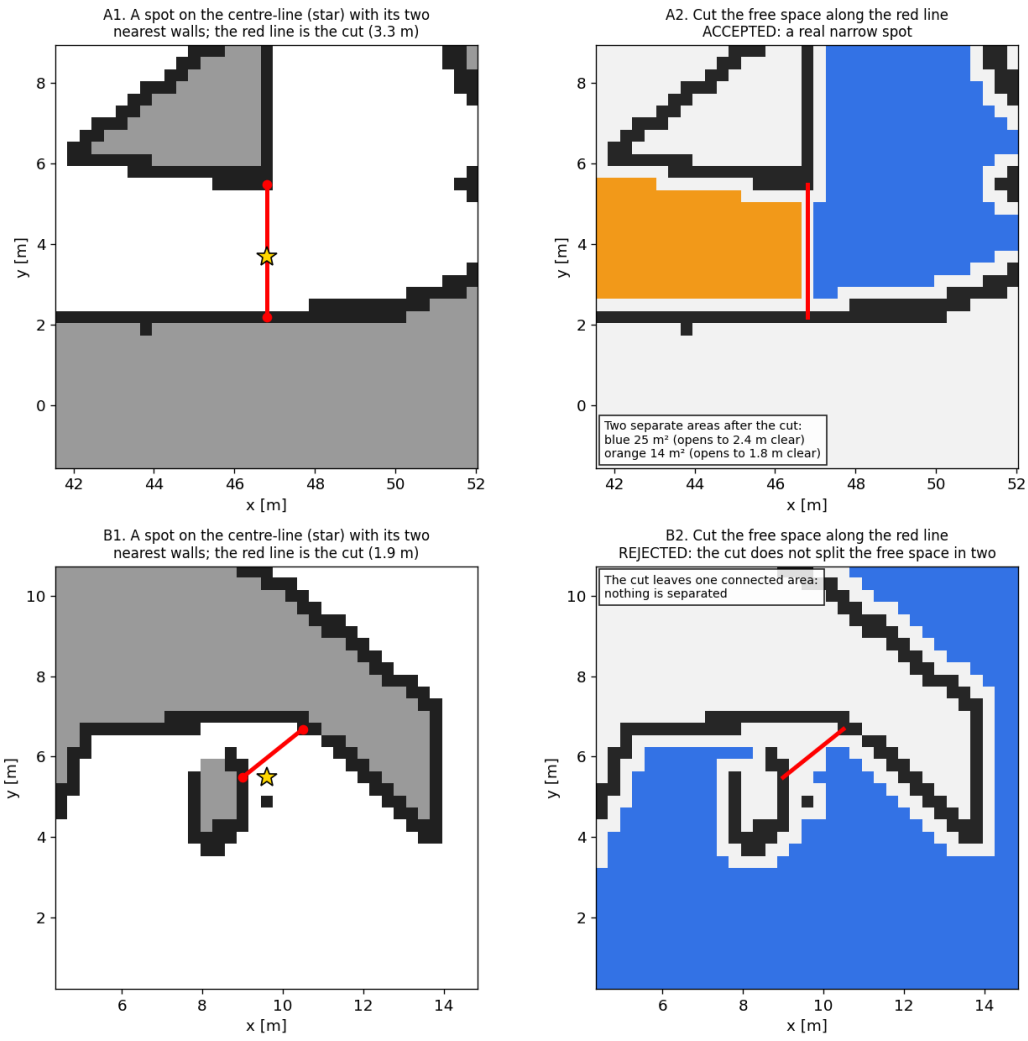


Figure 4: The cut test. Top: a 3.3 m gap, where cutting along the red line splits the free space into two separate areas (blue and orange), so it is accepted. Bottom: a small bump in a wall, where the cut leaves one connected area, so it is rejected.

not meaningful. The gap near $(6, 4)$ is missing from the world-file map. A likely reason is that the real laser also sees the green crystals standing beside it, which the wall-only drawing leaves out.

Speed. On the whole cave, one analysis takes about 0.02 s, so it keeps up with map updates.

7.6 Limits and possible improvements

- The cave is 3–5 m wide throughout, so "narrow" is relative. A gap is only judged narrow compared with the space beside it.
- Widths are only as exact as the map squares (0.3 m), so close results cannot be ranked reliably.
- A long corridor of constant width is reported at its mouth, not along its length.
- The analysis is not yet used by other parts. It could, for example, choose where to place sensors or relays, or serve as the skeleton of a roadmap (Advanced 4).

Table 2: What the cave geometry program reads and produces.

Name (ROS topic)	Direction	Type of data	What it holds
`/map`	In, from SLAM (about every 2 s)	Grid of whole numbers	One number per 0.3 m square: −1 not seen, 0 free, 100 wall. The map in Figure 5 is 216 × 170 squares, about 65 × 51 m
`cave_geometry/distance_transform`	Out, drawn in RViz	Grid of whole numbers	Same size as the map: the distance to the nearest wall, scaled so that 100 is the roomiest spot; −1 where the rover cannot go
`cave_geometry/markers`	Out, drawn in RViz	Shapes and text labels	Gold dots (centre-line), a cyan disc and label (widest area), red and orange lines with width labels (narrow spots)
Terminal messages	Out	Text	"Widest accessible area: centre, radius, area" and "Narrowest passage: place, width, kind, saddle", printed when they change

Table 3: The same analysis on two different maps of the same cave.

Finding	Map the rover built (SLAM)	Map drawn from the world file
Widest open area	Radius 5.7 m at (3.0, 33.1)	Radius 5.9 m at (2.3, 33.3)
Narrowest gap	3.1 m at (6.3, 4.3); the next is 3.3 m at (46.8, 3.7)	3.06 m at (47.3, 3.8)

8 Advanced 6 — Adaptive Scientific Sampling

8.1 Objective

Let the rover measure a spatially varying science quantity at its position, build a Gaussian Process (GP) model of the field from its own measurements, use the prediction and its uncertainty to choose where to sample next, and compare this with a simple baseline. The sampler must not use the true field when choosing.

8.2 The science data: real NASA measurements of water in Martian soil

What it is. To make the task realistic, we did not invent a made-up field. We used real measurements taken by NASA's *2001 Mars Odyssey* satellite, which has orbited Mars since 2001. One of its instruments, the *Gamma Ray Spectrometer*, detects the faint gamma rays that the ground gives off. Water changes this signal, so scientists can work out how much water is mixed into the top layer of the soil. The result is a map of Mars showing how wet the soil is in each place. We downloaded it from NASA's public Planetary Data System (data set `ODY-M-GRS-5-ELEMENTS-V1.0`, file `H2O_SR_5X5`; measurements made between June 2002 and April 2005) [1]. The way the values were derived is described in the paper by Boynton and colleagues [3].

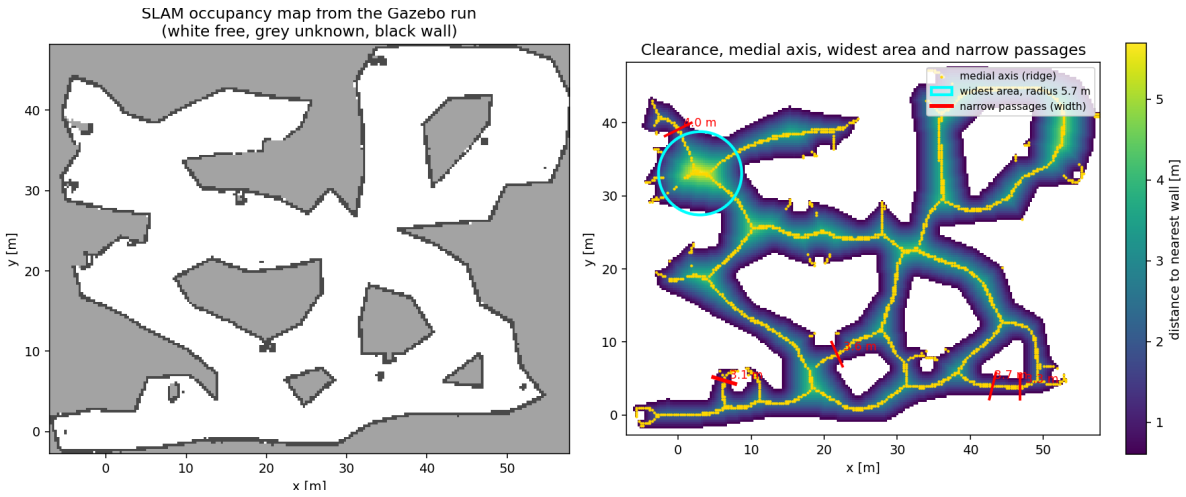


Figure 5: The final result on the real map from a Gazebo run. Right: gold is the centre-line, the cyan circle is the widest open area, and red lines are narrow spots with their widths.

Table 4: What each colour means in RViz.

Colour	Meaning
Gold dots	Centre-line of the cave (the spots furthest from the walls)
Cyan disc and text	Widest open area found so far, with its radius
Red line and text	The narrowest passage, with its width
Orange lines and text	Other narrow passages
Coloured overlay	Distance to the nearest wall: lowest next to the walls and highest in the middle of wide areas

How the amount of water is written. The amount is given as a *percentage of the soil's weight*. A value of 4 means that 4 kg of every 100 kg of soil is water. Over the whole planet the values run from 1.6 to 7.4. In the patch we use they run from 2.3 to 5.0.

What is in the file. NASA split the planet into a grid of squares, each 5° of latitude by 5° of longitude (about 300 km on each side). The file has 2592 squares, and each square is one line with five numbers. 1508 squares have a measurement. The rest, near the poles, are marked with the code 9999.999, meaning "no data".

8.3 How the data was put onto the cave map

The rover explores a cave about 65×55 m, so the NASA map had to be fitted onto it. This was done in five steps. The same steps are in the software, in `measurement_field.py`.

1. **Choose a piece of Mars.** We took a block of $12 \times 12 = 144$ squares around Arsia Mons (latitude 37.5°S to 17.5°N, longitude 212.5° to 267.5°E). Arsia Mons is a giant volcano, the southernmost of three in a row on the Tharsis bulge, a huge raised region near the equator of Mars. It lies at 8.35°S, 239.91°E [7], close to the middle of the block. We chose it because every square has a measurement, because the water varies from place to place (so there is something to discover), and because caves with openings in their roofs have been seen from orbit in this region.
2. **Match it to the map.** The cave map uses metres: x runs from -5 to 60 and y from -5 to 50. We stretched the block of Mars over this rectangle in a straight, even way:

$$\text{longitude} = 212.5 + \frac{x+5}{65} \times 55, \qquad \text{latitude} = -37.5 + \frac{y+5}{55} \times 55 \;=\; y - 32.5.$$

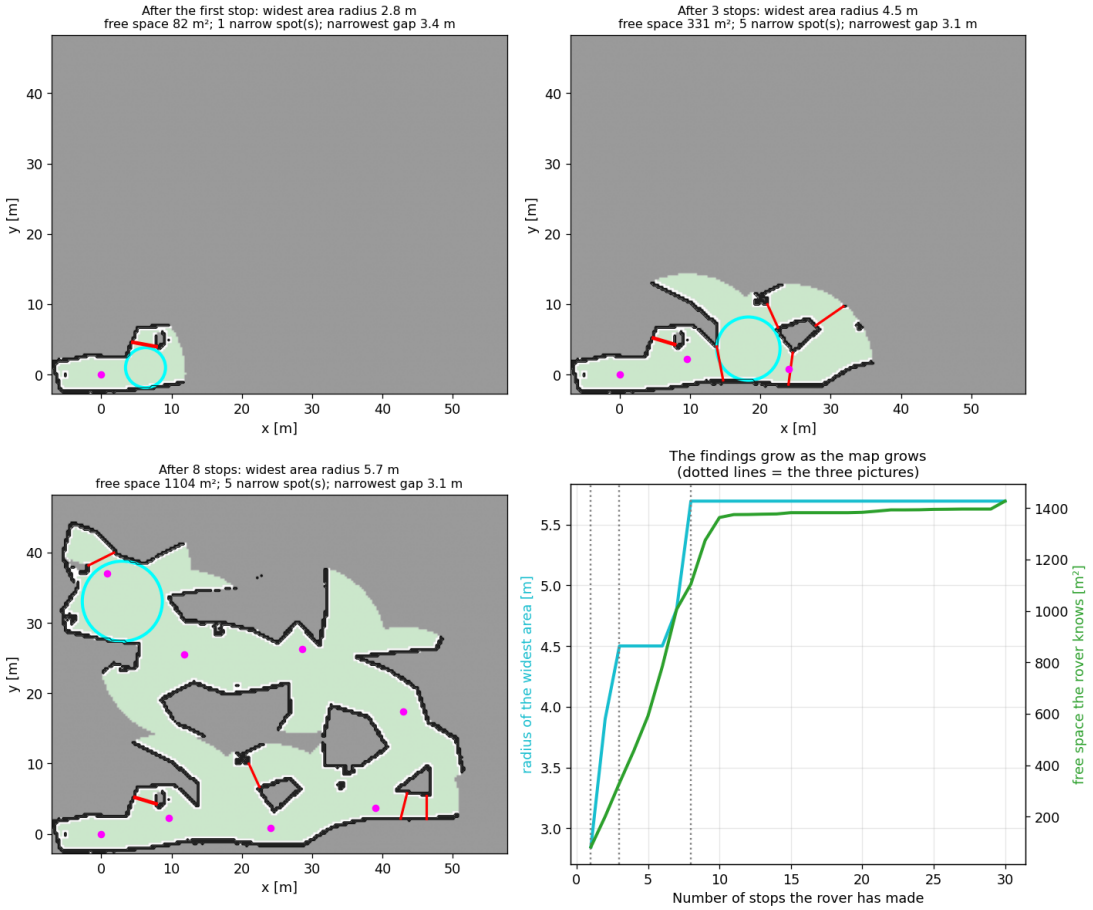


Figure 6: The result after the rover's first, third and eighth stops (pink dots), and a graph of all stops. Green is the space known so far, the cyan circle is the widest area, and red lines are narrow spots. This replays the run's real stops over its final map.

For example, the rover's start $(x, y) = (0, 0)$ is placed at 32.5°S, 216.7°E. A position outside the rectangle uses the nearest edge.

3. **Make it smooth.** A square has a single number, but the rover can stand anywhere. Between the middles of neighbouring squares we blend the numbers in a straight line, so the water changes smoothly and has no sudden steps.
4. **Give each place its true value.** Now every position the rover can reach on the map has a *true* water value. This is kept hidden from the part of the program that decides where to go.
5. **Make readings imperfect.** A real instrument is never exact. When the rover takes a reading, the program returns the true value plus a small random error of the size NASA reports for that square (0.23 to 0.36 inside our block, 0.27 on average). Two readings at the same place therefore differ slightly.

The scale is not realistic. One square on Mars is about 300 km across, and our block is about 3250 km wide, roughly the distance from Sydney to Perth. We squeeze all of that into a cave tens of metres wide, so one metre in the cave stands for about 50 to 60 km on Mars. The task does not need the numbers to match a real cave, only a believable field that varies from place to place.

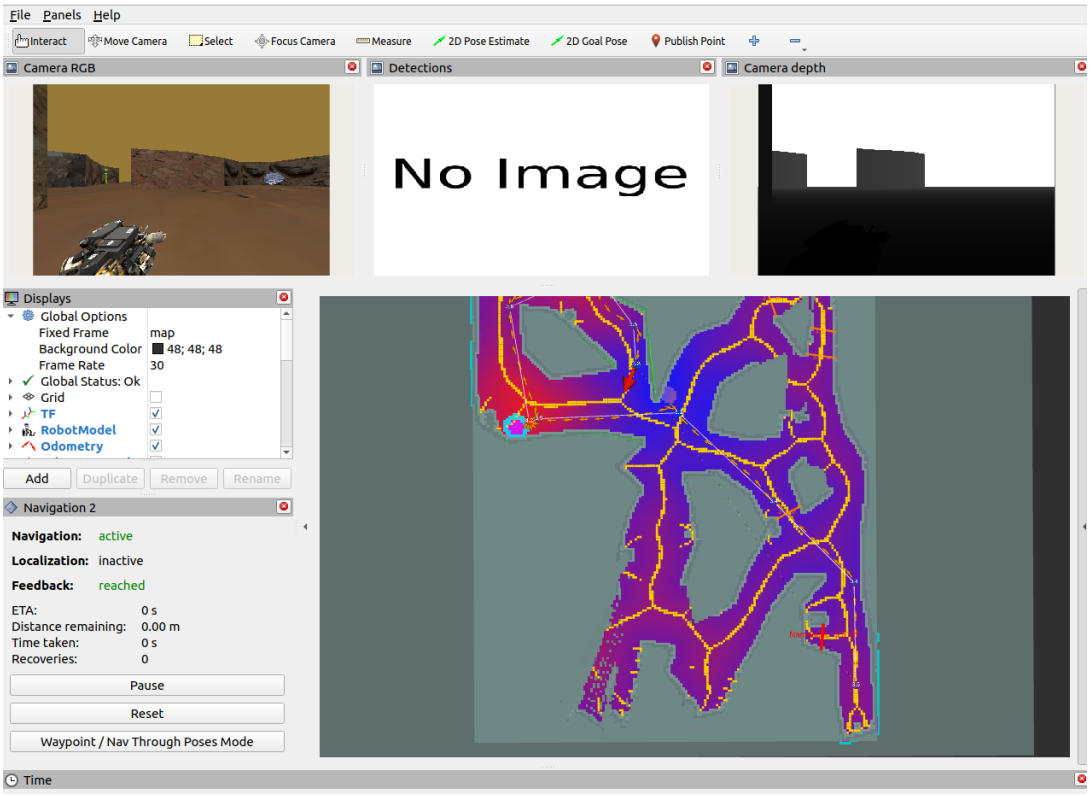


Figure 7: RViz during a Gazebo run (screenshot taken while the first, 10-reading run was in progress). The map panel shows the cave geometry overlay: the colour shows the distance to the nearest wall (red = the most room, as in the large chamber at the top left, blue = close to walls), the gold lines are the centre-line, and the red line and label near the bottom right mark the narrowest passage. The white lines join the sampling spots in the order visited, with the readings written beside them (Advanced 6). The Detections panel is empty because the node that fills it was not running in this test.

Where the data is in the simulator. The water values are not objects that you can see in Gazebo's 3D view. They exist inside the software as a lookup: the program is given a position on the map and returns the water value there. They are drawn for people in RViz, on the same map the rover is building (Section 8.4).

8.4 What the data looks like in a simulator run

Figure 9 comes from one real run in Gazebo: the rover took 30 readings with real navigation. The first panel shows the 30 places it visited, in order, on top of the true field. The second shows what the rover worked out about the whole cave from only those 30 readings. The third shows how unsure it still is. The fourth compares each reading with the true value at that spot. The points lie close to the dashed line, so the readings are close to the truth. The average mistake was −0.03 and the typical size of a mistake was 0.21, which matches the instrument error that NASA lists for this block (0.23 to 0.36). The rover's overall picture of the cave (panel 2) was off by 0.23 on average. The brightest part of panel 3, in the top left of the cave, is the area the rover has measured least.

8.5 The real instrument and our virtual one

The real instrument. The water map was made by the *Gamma Ray Spectrometer* on Mars Odyssey, a satellite that has circled Mars in a nearly circular orbit about 400 km above the

Table 5: What each line of the NASA file contains, with a real example. The example square is the one that contains the rover's start position in the simulator.

Column	Meaning in plain words	Unit	Real example
Latitude	How far north (+) or south (−) of the equator the middle of the square is	degrees	−32.5
Longitude	How far east of a reference line the middle of the square is	degrees east	217.5
Concentration	Amount of water in the soil: the measurement itself	% of soil weight	3.609
Error (σ)	How uncertain NASA's measurement is: a typical size of its mistake	% of soil weight	0.268
Error with corrections	The same, after allowing for the corrections NASA applied (not used by us)	% of soil weight	0.320

Table 6: How each part of the NASA file is used in the simulator.

In the NASA file	In the simulator	What it is used for
Latitude and longitude of a square	A place on the cave map, in metres (x, y)	Says where on the map each value applies
Concentration (water %)	The true water value at that place	What the rover is trying to find out. Hidden when the rover chooses where to go; used afterwards to mark its work
Error (σ)	The size of the random error added to each reading	Makes readings imperfect, like a real instrument
"No data" squares (9999.999)	Not used	The block we chose has none

surface [2]. It is really a set of three instruments that work together: a gamma-ray spectrometer, a neutron spectrometer and a high-energy neutron detector [4]. They cannot see water directly. Cosmic rays from space hit the ground and knock out neutrons, and those neutrons make the soil give off gamma rays whose energies are a fingerprint of the elements present. Hydrogen leaves a clear mark, and because most hydrogen on Mars is locked in water, scientists turn the hydrogen signal into a water amount [5]. The instruments sense only the top layer of the ground, the upper few tens of centimetres [2]. From orbit, one measurement covers an area hundreds of kilometres across, which is why NASA's file uses squares 300 km wide.

What we built instead. We did not simulate the detector, the gamma rays or the neutrons. We simulated what a user of the instrument gets out of it: a water value, with an error, for a place. The rover gives its position, the program looks up NASA's water value for that place (blending smoothly between squares), adds a random error of the size NASA reports for that square, and returns the answer. In the code this is the function `measure(x, y)` in `measurement_field.py`. The part of the program that chooses where to go can call only this function, and never sees the true value.

What the simulation leaves out. It leaves out the physics of the gamma rays and neutrons, the blurring caused by the 300 km footprint (ours reads a single point), and any error that is shared between measurements. Our errors are all random and independent of each other, but real errors are partly systematic. The nearest real equivalent for a rover is the neutron instrument on the Curiosity rover, called DAN, which looks for hydrogen under the wheels [6]. We did not

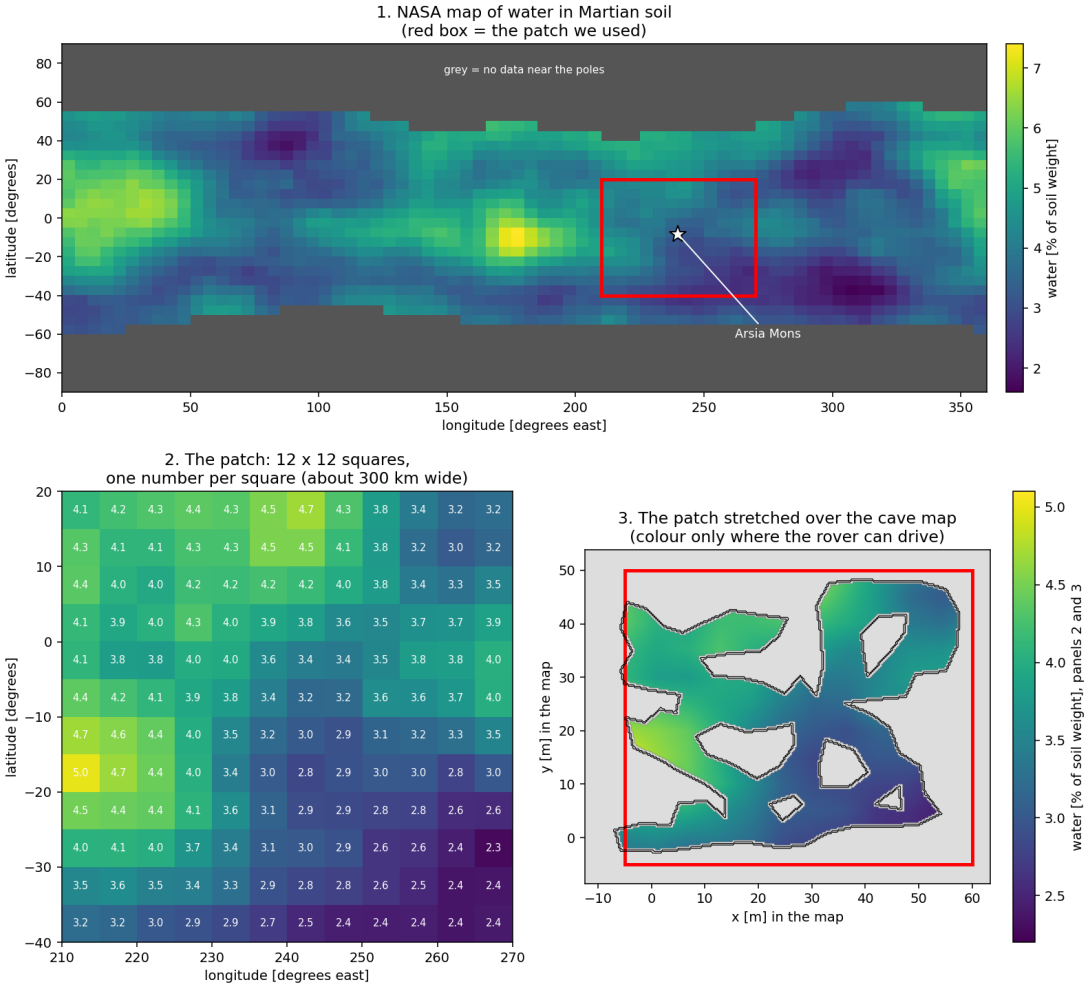


Figure 8: From NASA's map to the cave map. (1) The whole NASA map of water in Martian soil; the red box is the block we used, and the star marks Arsia Mons, a giant volcano near the middle of the block. (2) The block itself: 144 squares with their values. (3) The same block stretched over the cave map. Colour is shown only where the rover can drive. Panels 2 and 3 share one colour scale.

model it.

8.6 What is real and what is not

The task description allows this: it asks for a field that "does not need to realistically match the cave environment" as long as it gives a believable, varying quantity to measure.

8.7 Method

Separation of truth. The sampler (`adaptive_sampling.py`) has no reference to the field. It receives one measurement at a time. The true field is read in one place, to score the result.

Gaussian Process. scikit-learn's `GaussianProcessRegressor`, input the robot's (x, y), a constant $\times$ RBF kernel plus a white-noise term, targets normalised. The length scale is bounded to 5–60 m. The hyperparameters are fitted only from 8 samples; before that the prior (length scale 10 m) is used, because the fit on fewer samples chose a very short length scale that made everywhere look equally uncertain.

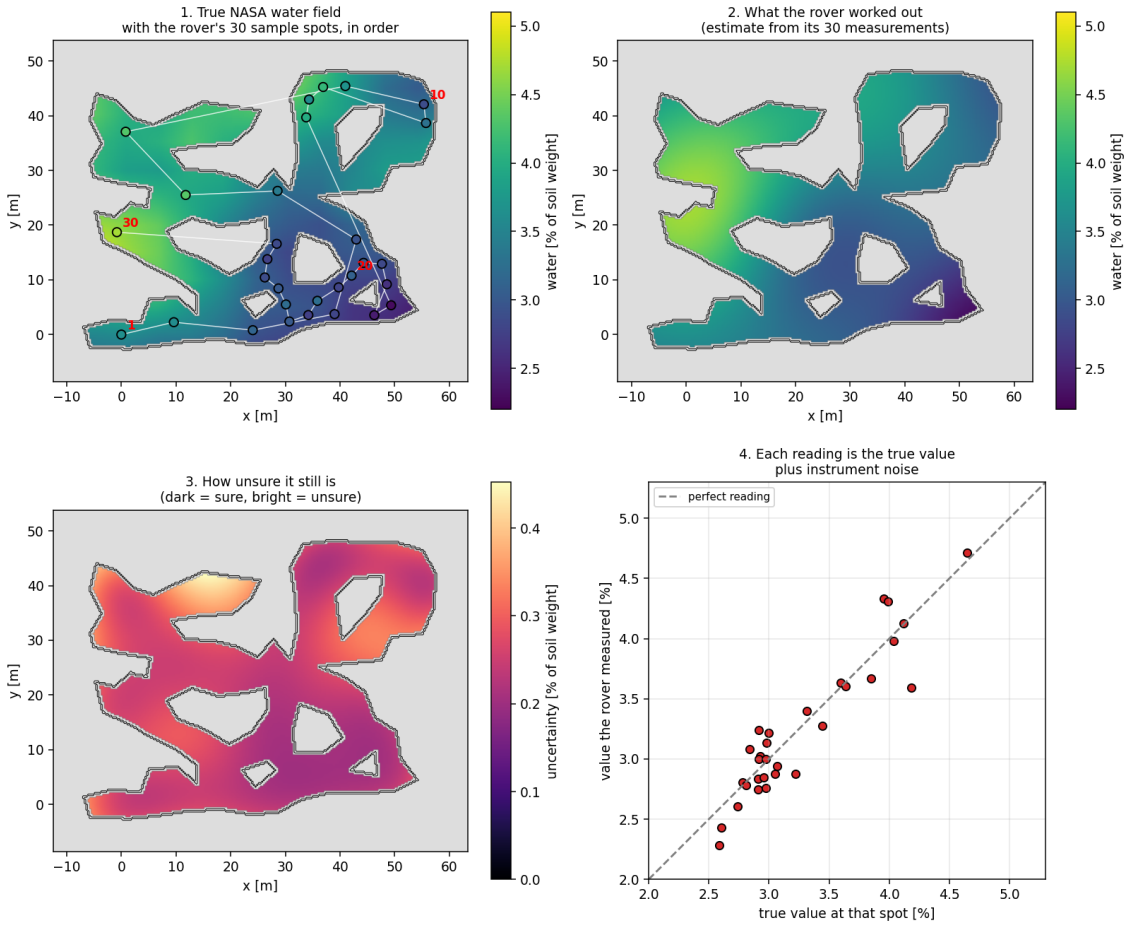


Figure 9: One 30-reading run in Gazebo. (1) True field, with the places the rover measured, in order. (2) The rover's estimate. (3) Its remaining uncertainty. (4) Each reading against the true value. The estimate was re-computed from the readings saved by the run.

Choosing the next location. Candidates are cells of a 1 m lattice in the accessible part of the current map, at least 1.5 m from walls and unknown space (so Nav2 can reach them), at least 4 m from existing samples and at least 4 m from goals that failed. Each is scored by

$$\text{score} = \frac{\sigma(\mathbf{x})}{\max \sigma} \; - \; w \, \frac{\text{path}(\mathbf{x})}{\max \text{path}}, \tag{1}$$

the GP standard deviation minus a penalty for the shortest path through free space (Dijkstra on the grid, not straight-line distance). The best score is the next goal. The weight w is the *travel weight*; 0.3 is the default and 0.1 was also tested. The baseline picks uniformly at random from the same candidates. Because candidates exist only in the known map, the method also drives exploration toward its edge.

```
_, std = self.predict(xy)
score = (std / max(std.max(), 1e-9)
         - self.travel_weight_ * distances / max(distances.max(), 1e-9))
best = np.argmax(score)
```

Listing 2: Choice of the next sampling location (adaptive strategy).

Driving the robot (`adaptive_sampling_node.py`). The node measures at the robot's position, updates the GP, chooses the next point and sends it to Nav2, then repeats. It publishes

Table 7: The first eight readings of that run. "Reading" is what the rover measured; "True value" is the NASA value at that spot before any error was added.

No.	Place in the cave map (x, y) [m]	Place on Mars (latitude, longitude)	True value [%]	Reading [%]	Difference
1	(0.0, 0.0)	32.5°S, 216.7°E	3.60	3.63	+0.03
2	(9.6, 2.3)	30.2°S, 224.9°E	3.64	3.60	−0.04
3	(24.0, 0.8)	31.7°S, 237.1°E	2.98	3.14	+0.16
4	(39.0, 3.7)	28.8°S, 249.7°E	2.78	2.80	+0.02
5	(43.0, 17.4)	15.1°S, 253.1°E	3.07	2.94	−0.13
6	(28.6, 26.3)	6.2°S, 240.9°E	3.31	3.40	+0.09
7	(11.8, 25.5)	7.0°S, 226.7°E	3.95	4.33	+0.38
8	(0.8, 37.1)	4.6°N, 217.4°E	3.99	4.31	+0.32

Table 8: The real instrument compared with our simulated one.

	Real instrument (Mars Odyssey)	Our simulated instrument
Where it is	On a satellite, about 400 km above Mars	On the rover, at its position
What it covers	An area about 300 km wide per measurement	A single point
How a value is made	Counting gamma rays, then turning the hydrogen signal into water	A look-up in NASA's finished map
Where the error comes from	Few gamma rays counted, and corrections for other effects	A random number of the size NASA reports
What it returns	Water as a percentage of soil weight	The same

the GP mean, the GP uncertainty, the true field and the samples (coloured by value) to RViz, and writes a CSV of every run. It runs instead of the exploration node, since both send goals. Two faults found in simulator runs were fixed:

- **Instant aborts.** Nav2 aborts a goal that arrives the moment the previous one ends. The node treated each as a real failure and excluded a 4 m circle around a perfectly good place (30 times in one run, sending the rover zig-zagging, 648 m for 30 samples). Goals aborted within 3 s are now retried, after a 1 s pause.
- **Wedged rover.** A goal placed beside an artefact in a narrow passage left the rover stuck for 12 minutes while the node waited out 120 s timeouts. The node now detects no movement for 15 s, cancels the goal, reverses for 4 s, turns right for 2 s, and avoids that place. Goals were also moved from 0.9 m to 1.5 m from walls.

8.8 Results

Offline study (no simulator). The robot is moved instantly, the map is revealed within 12 m of each place visited, and the error is the RMSE over the whole reachable cave. Ten runs per strategy, 40 samples each (Figure 10).

Gazebo runs (real Nav2 and SLAM). Each run takes 30 readings after a fresh simulator start, with real navigation and a real SLAM map. Each setting was run with four seeds (0 to 3), giving 12 runs in all. A seed fixes the random measurement errors and, for the random strategy, the random choices, so the three settings face the same conditions. The tables below are generated from the saved run logs.

Table 9: Which parts of the data are real, which we adapted, and which are not real.

Real	The water values, where Mars is wetter and drier, and the size of the instrument's error. All of these come from NASA's file.
Adapted by us	The choice of block; stretching it over the cave; blending smoothly between squares; adding a random error to each reading (the size of the error is NASA's, the random numbers are generated by the computer).
Not real	That the water in a cave would vary like this; the scale (one metre stands for 50–60 km); and the idea that the rover has a water instrument at all. Also, NASA's file does not say whether longitude counts east or west. We assumed east, after checking that the known wet regions appear in the expected places.

Table 10: Offline comparison: RMSE of the reconstructed field (wt% H_2O), mean ± standard deviation over 10 runs.

Samples	Adaptive	Random
5	0.567 ± 0.077	0.483 ± 0.042
10	0.463 ± 0.088	0.486 ± 0.096
20	0.293 ± 0.031	0.366 ± 0.098
30	0.250 ± 0.044	0.250 ± 0.030
40	0.215 ± 0.044	0.217 ± 0.024
Mean distance travelled (40 samples)	476 m	934 m

How to read the results.

- **Adaptive drives clearly less.** With travel weight 0.3, every adaptive run drove less than every random run (341 to 555 m against 670 to 818 m). On average that is 477 m against 735 m, about 35% less distance, and about 35% less time (10.5 against 16.1 minutes).
- **Final accuracy is about the same.** After 30 readings the average error is 0.228 for adaptive (weight 0.3) and 0.237 for random. Seed to seed, the error varies by about 0.04, which is larger than the difference between the two strategies, so these results cannot show that one is more accurate than the other.
- **Early accuracy is only a hint.** After 10 readings adaptive averages 0.34 and random 0.44, but one adaptive run (seed 3) was as bad as the worst random run at that point (0.54 for both). With four seeds this is not reliable.
- **Reaching a given error takes the same driving.** The average distance at which the error first dropped to 0.30 was 168 m for adaptive and 172 m for random. Adaptive's saving comes after that point: it keeps improving without long trips, while random keeps crossing the cave.
- **A smaller travel weight did not help.** With weight 0.1 the final error was slightly lower (0.197) but within the spread, and the robot drove about 30% farther (625 against 477 m). Weight 0.3 is the better choice.

In short: in these runs the adaptive method gives a survey of about the same quality for about two-thirds of the driving and time. It did not give a more accurate one. Four seeds is a small sample, so a smaller difference could still be hiding in these numbers.

[TODO: Karan: optional: add an RViz screenshot showing the GP mean, uncertainty and true-field overlays, which the screenshot in Figure 7 predates. The sample markers are visible there.]

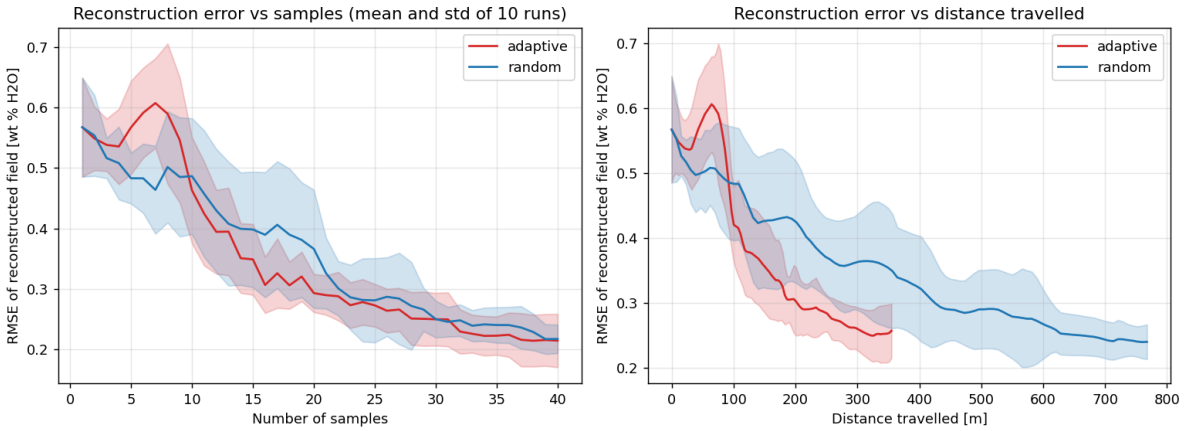


Figure 10: Offline reconstruction error against samples (left) and distance travelled (right).

Table 11: Gazebo comparison: RMSE of the reconstructed field after 10, 20 and 30 samples (mean ± std over seeds), mean distance and wall-clock time for 30 samples, and mean distance at which RMSE first reached 0.30 (number of runs that reached it).

Strategy	Runs	RMSE @10	RMSE @20	RMSE @30	Distance	Time	Dist. for RMSE≤0.30
Adaptive, travel weight 0.3	4	0.342 ± 0.113	0.262 ± 0.031	0.228 ± 0.039	477 m	10.5 min	168 m (4/4)
Adaptive, travel weight 0.1	4	0.323 ± 0.080	0.261 ± 0.035	0.197 ± 0.036	625 m	14.3 min	184 m (4/4)
Random baseline	4	0.435 ± 0.069	0.250 ± 0.026	0.237 ± 0.035	735 m	16.1 min	172 m (4/4)

8.9 Discussion and limits

- In the offline study, adaptive is worse than random after 5 samples, because the travel penalty keeps it near the start while the GP extrapolates over unseen cave. It is ahead from about 10 samples (clearest at 20: 0.293 against 0.366) and uses about half the distance. By 30 and 40 samples the two have the same error (0.250 against 0.250, and 0.215 against 0.217). The reliable advantage is therefore less driving, not a lower final error. The Gazebo runs below point the same way.
- The offline study ignores driving time and navigation failures. The Gazebo runs include them.
- Measurement noise (0.23 to 0.36 wt% inside the block) is large against the field's range (2.3 to 5.0 wt%), which puts a floor on the achievable error of about 0.2.
- The online error is computed over the cave explored so far, so its region grows during a run.
- Possible improvements: choose the travel weight from the remaining sample budget; use the map to sample at ridge points rather than a lattice; and a kernel that is not isotropic.

9 Results

STUDENT ACTION REQUIRED — all

Describe what was achieved overall, with pictures, graphs or plots as evidence of a working system. Include the video link below. The report lead will assemble this once all parts are on the shared repository and have been run together.

Summary of what has been shown, by part.

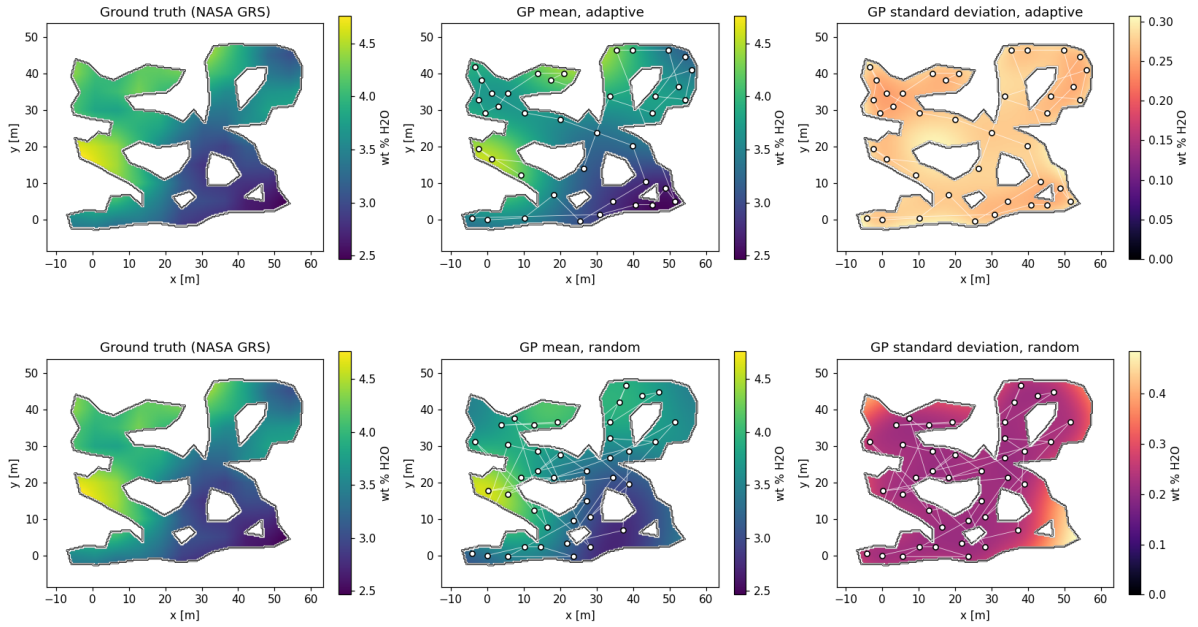


Figure 11: One offline run per strategy. Left: true NASA field. Middle: GP mean. Right: GP standard deviation. White dots are samples joined in visiting order.

Table 12: Every Gazebo run: final error (RMSE after 30 readings) and distance driven. Lower is better.

Seed	Adaptive, weight 0.3	Adaptive, weight 0.1	Random
0	0.233 / 341 m	0.140 / 754 m	0.180 / 818 m
1	0.189 / 480 m	0.191 / 586 m	0.271 / 726 m
2	0.201 / 555 m	0.230 / 576 m	0.258 / 670 m
3	0.290 / 532 m	0.226 / 582 m	0.237 / 725 m

- **Advanced 2:** widest area and passages found and displayed on the real SLAM map (Figure 5).
- **Advanced 6:** 12 runs of 30 readings completed in Gazebo with real navigation (4 seeds, 3 settings); offline comparison over 10 runs (Section 8).
- **Perception, Planning:** **[TODO: Andrew and Christina: one or two lines each, with a figure.]**

Video. **[TODO: Link to the video(s). Upload to YouTube, OneDrive or Google Drive and check that the marker can open it. If several short videos are submitted, say so here and list them. About five minutes in total.]**

10 Reflection on Teamwork

Write this from your own experience

The points below are facts known from the repository history; the marker rewards a specific, honest account over a polished generic one. Replace the brackets with what actually happened.

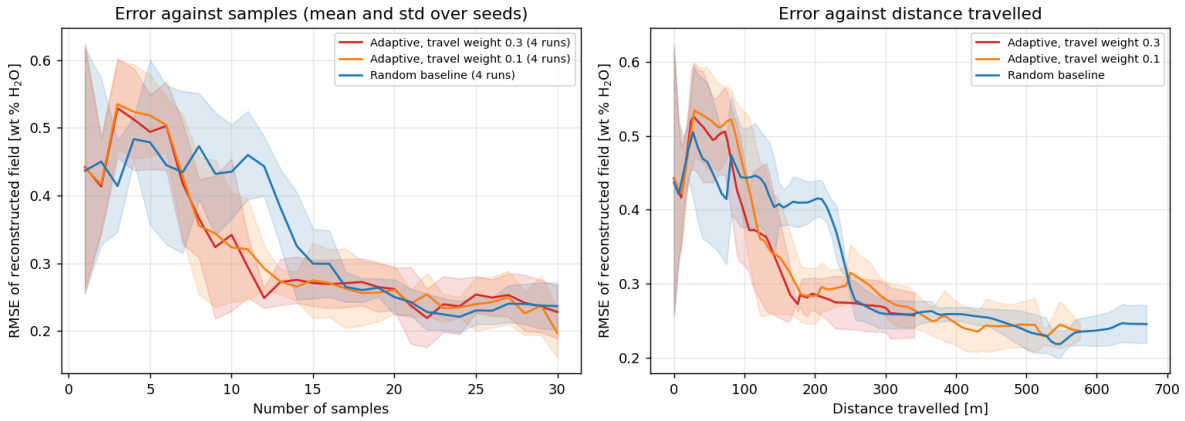


Figure 12: Gazebo runs: error against samples and against distance travelled.

- **Plan.** The team agreed on a split by task block (Perception, Planning, Advanced), with dated checkpoints and a list of what each member owes the others. The repository shows the first contributions from Perception on 11 October and from Planning on 12 October, after the first planned checkpoints (6 to 10 October).
- **Shared repository.** One member committed straight to the main branch and edited the shared node, while another rewrote the same file on a separate branch, so the two versions conflict and have to be merged by hand. Karan kept the Advanced work in separate nodes and branches so as not to edit the shared file. The two versions had not been merged at the time of writing.
- **Dependencies.** The Planning behaviour needs the positions of artefacts from Perception. These were not on the repository by 12 October, so Planning uses a fixed range of 3 m along the direction of the camera as a stand-in.
- **Integration.** The parts had not been run together at the time of writing.
- **What each member learned.** **[TODO: One or two sentences each, in your own words.]**

References

[1] W. V. Boynton. *2001 Mars Odyssey Gamma Ray Spectrometer Element Concentration Data*. NASA Planetary Data System, data set ODY-M-GRS-5-ELEMENTS-V1.0, 2007 (product H2O_SR_5X5). This is the citation given in the data set's own catalogue. https://pds-geosciences.wustl.edu/ody/ody-m-grs-5-elements-v1/odgm_xxxx/. Downloaded 5 October 2026.

[2] NASA Planetary Data System. Catalogue files of the same data set (`inst.cat`, `mission.cat`): instrument description, mapping orbit of 400 km, and sensing depth of the upper few tens of centimetres. Read 12 October 2026.

[3] W. V. Boynton et al. "Concentration of H, Si, Cl, K, Fe, and Th in the low- and mid-latitude regions of Mars." *Journal of Geophysical Research: Planets*, 112(E12), 2007. doi:10.1029/2007JE002887.

[4] W. V. Boynton et al. "The Mars Odyssey Gamma-Ray Spectrometer instrument suite." *Space Science Reviews*, 110(1–2), 37–83, 2004. doi:10.1023/B:SPAC.0000021007.76126.15.

[5] W. C. Feldman et al. "Global distribution of near-surface hydrogen on Mars." *Journal of Geophysical Research: Planets*, 109(E9), 2004. doi:10.1029/2003JE002160.

[6] I. Mitrofanov et al. "Dynamic Albedo of Neutrons (DAN) experiment onboard NASA's Mars Science Laboratory." *Space Science Reviews*, 170(1–4), 559–582, 2012. doi:10.1007/s11214-012-9924-y.

[7] "Arsia Mons". Wikipedia. https://en.wikipedia.org/wiki/Arsia_Mons. Position 8.35°S, 239.91°E. Read 12 October 2026.